The Relay: write-up

Archi Kanungo · Code, raw results (`results/results.csv`) and decision log: github.com/archiik04/the-relay

1. What I built and why

A Python assistant that answers only from the three pinned papers (exact arXiv versions, stamp-checked). Five steps:

1. **Understand.** Rules, no model call: which papers are named, and is it a comparison?
2. **Retrieve.** A `search_papers` tool over 384-token section-aware chunks (BM25 + bge-small, fused), top 4, with a timeout and one retry. A two-paper question searches each paper separately (the branch).
3. **Draft.** One gpt-oss-20b call returns JSON claims, each with a chunk ID and an exact quote.
4. **Check.** Code verifies every quote word for word in its chunk; one redraft (the loop), else refuse.
5. **Send.** The reply is built *only* from checked claims, with citations from chunk metadata, and nothing is sent before the check.

Every model call goes through one **gateway**: token buckets that refill continuously like Groq's, a bounded FIFO queue, a 10 s admission deadline, capped retries, and a labelled passages-only reply when the daily budget is gone.

The decision that shaped it most: one model call per answer. Groq allows 8,000 tokens/min and reserves prompt + max_tokens (~2,500) per call: about 3 answers a minute. A second model call to check the answer would halve that, so the check is code. §8 gives the measured price. **"Fast"** means a reply starts within ~1 s when idle (p50 945 ms, n=18), no admitted request queues more than 10 s, and anyone we can't serve in time is told at once.

2. The real service and my stand-in

I found the limits by sending tiny calls until a 429 (30/min), large prompts until a 429 (8K tokens/min), and 10 calls at once (all succeeded), and by reading `x-ratelimit-*` on every response (1,000 requests/day). Surprises:

- **Limits refill continuously** (one request per 86.4 s), not at midnight.
- **Admission uses prompt + max_tokens**, but the final charge is the tokens actually used.
- **A 429 costs no daily request.**
- **JSON mode can fail server-side** inside a 200 stream, returning nothing.
- Tokens/day was never reached (documented 200K, low confidence).

Stand-in: a service behind the same interface, calibrated by resampling **70 real calls**: first token p50 655 ms, generation p50 186 ms, 168 output tokens. Like Groq, it 429s in ~65 ms with retry-after and the same headers, streams in pieces, fails at the measured JSON rate, has slow outliers, and slows as calls pile up.

Where it differs most:

- It runs in-process, so there are no network delays. Real calls showed seconds before Groq's queue (§4).
- Its slowdown under load is assumed: Groq barely slowed at 10 calls at once.
- Its answers are synthetic, so answer quality is "not measured" on stand-in rows.

3. Results

Tables 1–3 follow, all computed from `results.csv` by `scripts/analyze_results.py`: nearest-rank percentiles with n shown, first-token times for requests that got a reply, and real vs stand-in marked per row.

A number we could not have guessed: Groq's JSON mode rejected **1.25%** of outputs server-side, every time on the same maths-heavy question (Q03).

Answer quality (18 self-written questions):

- **Refusals: 5/6 correct.** All near-miss and off-topic questions were refused; the false premise was refused but not corrected.
- **Answerable: 4/12 correct by the automatic grader, 3/12 by my hand review**, at ~2,100 tokens per answer.
- **Hand labels: 50% of claims (10/20) say more than their own quote.** The grader matched my verdicts on 13/18.

4. Where the time goes

One slow request: Q04 (real), first token at 3,518 ms.

- Working: retrieval 24.5 ms, then generation of 212 tokens (132 reasoning) in 226 ms, then the quote check in 1.9 ms.
- Waiting: 3,265 ms before the model's first token.
- Retrying: none.

Groq's own `usage.queue_time` explains only 368 ms of the wait. **The delay came before the first token, and mostly outside Groq's reported queue** (network or front-end, unresolved). We know from four timestamps per request (`model_first_token_ms`, `model_last_token_ms`, `first_token_ms`, `total_ms`) plus Groq's usage fields. Buffering the reply for the check cost 1.9 ms.

5. When a limit is hit

Per-minute.

- The gateway estimates the wait from the queue and the token refill rate. If it's ≤ 10 s, the request waits in FIFO order. Otherwise the person hears "busy, try again in ~N s" within 20–50 ms.
- Everyone is delayed by at most 10 s; whoever arrives once that 10 s is used up is turned away.
- A 429 is retried after its retry-after, only if the deadline allows.
- When the limits were halved without notice, the gateway learned it from 429s and headers: 70% still succeeded, p95 6.9 s.
- A 25 s deadline answered as many people under overload, but at p95 24.1 s instead of 10.8 s, so we chose 10 s.

Per-day.

- No model call, and no fallback model: that would dodge the limit.
- The reply is a **labelled passages-only answer**.

- Because the limit refills one request every 86.4 s, full answers trickle back (one at +135 s in our run).
- Nobody gets a weaker *generated* answer.

Retrieval: quality versus cost

- **Table 3: [TOMORROW: bm25 and dense rows]**. Retrieval alone (12 answerable questions, top 4): hybrid found a gold page for 91.7% (MRR 0.833, 19 ms), dense 100% (0.840, 16 ms), bm25 91.7% (0.708, 0.8 ms). A cross-encoder reranker never helped and cost +600–770 ms of CPU.
- **Choice: hybrid, no rerank.** BM25 catches exact tokens ("Table 1", "28.4") and dense catches paraphrase, for 19 ms of a ~1 s request. With 12 questions, one question is 8.3 points.
- **Near the limit we don't shrink retrieval.** We only degrade to benchmarked settings, so we stop at the passages-only reply instead.

How this library will mislead a naive retriever

1. **Comparisons pull one paper.** For Q12 (Transformer positions vs H-Net boundaries), a single search returned only H-Net passages in every setting. Fix: one search per paper.
2. **Shared vocabulary.** For Q04 (Moshi's *Depth Transformer*), "Transformer" pulled the Attention paper's heads-and-layers table (p.9) into the top 4. Moshi's own table puts the Temporal Transformer's 32/32 next to the Depth Transformer's 16/6.
3. **Facts lost in extraction.** H-Net's 0.5 boundary threshold lives only in Equation 4, which extraction drops (Q09). Attention Table 2 merges rows, so four models share three BLEU values.

6. Retries

Policy.

- Retry a 429 after its retry-after: at most 2 per request, 20% of traffic, within the deadline.
- A retry-after longer than the deadline means the daily limit, so stop calling.
- Redraft a failed check once.

Where retrying made things worse. In real run 1, three redrafts (Q02, Q03, Q10) each paid a second full call (~2,500 tokens) and still failed. Q10 returned the identical draft. At 8K tokens/min, each wasted call costs ~21 s of budget: one other person's answer.

Where my expectation was wrong. On the stand-in under overload, blind 429 retries beat our gateway: 21 vs 19 answered, using 297 calls instead of ~62. A 429 is cheap: it costs no daily request and takes 60 ms. Lesson: retry what is temporary; don't repeat what is deterministic.

7. What the person sees

At 3 pm, with the daily budget gone, they get this within ~30 ms:

> *[Limited answer] Today's answering budget is used up, so I can't write a checked answer now. Full answers resume in about N minutes. These are the most relevant passages I found; they may not answer your question:* followed by three cited passages, e.g. *[hnet p.6] 2.2.1 Chunking Layer: "…"*

Every passage is real and cited, the reply is honest about the limit, and nothing is guessed.

8. What I'm unsure about, what I left out, what I learned

Least certain assumption: that a verified quote means a supported claim. It doesn't: **50% of claims overstate their quote**. If that holds, users read confident sentences with real citations that claim more than the source says. The fix is a per-claim support check, either a second model call (half the throughput) or a stricter local rule.

Two things I deliberately did not build:

1. A separate HTTP stand-in server: in-process, under deadline.
2. Priority classes and circuit breakers: there's no priority data, and a 429 already says when to retry.

The 20b vs 120b comparison was also cut. **New to me: [ARCHI]**.

Your feedback on Vaani

[ARCHI]

How I worked with the assistant

[ARCHI] Real conversation history: `conversation-history` file in the submission.

- **Three prompts that mattered: [ARCHI]**
- **Got wrong, and I caught it: [ARCHI: e.g. its version check passed v10 as v1, found via my "v6 test" explanation; its table splitting silently dropped text, exposed by my "no truncation" rule.]**
- **Where I overruled it: [ARCHI: e.g. no fallback model; quotes ≠ support; the 10 s deadline.]**
- **Distinctly mine: [ARCHI]**

Results tables (required by the brief)

Generated by `scripts/analyze_results.py` from `results/results.csv` only; every number below can be recomputed from that file. Times in ms; percentiles nearest-rank, n shown.

Table 1: Limits found in the real service (computed from probe-* rows)

Limit	Value found	How found	Confidence
Requests per minute	30 OK, then a 429	probe-rpm: tiny calls back to back until the first 429	high
Tokens per minute	8000 (header); 429 after 3 large calls	x-ratelimit-limit-tokens on every response + probe-tpm	high
Requests per day	1000 (header)	x-ratelimit-limit-requests; remaining drops by 1 per call	high
Tokens per day	not observed (documented 200K)	never reached; no header reports it	low
Requests in flight at once	10 of 10 succeeded at once (626–787 ms)	probe-concurrency	medium (only 10 tried)
What it returns at a limit	HTTP 429 in 60 ms, retry-after 2 s (2 429s seen)	probe rows with status rate_limited	high
Time until it recovers	2,072 ms (TPM), 3,180 ms (RPM) after the first 429	next successful call after the 429	medium (one episode each)

Table 2a: Latency

Scenario	Real or stand-in	Requests sent	Duration (s)	Offered rate (per min)	First token p50 / p95 / p99 (ms)	Total time p50 / p95 / p99 (ms)
burst	stand-in	12	2	12 at once	1,187 / 1,981 / 1,981 (n=3)	1,187 / 1,981 / 1,981 (n=3)
daily_exhausted	stand-in	17	155	6.6	38 / 2,011 / 2,011 (n=17)	38 / 2,011 / 2,011 (n=17)
overload	stand-in	62	304	12.6	6,531 / 10,762 / 10,762 (n=19)	6,531 / 10,762 / 10,762 (n=19)
overload_naive_retry	stand-in	62	299	12.6	2,656 / 6,372 / 6,678 (n=21)	2,656 / 6,372 / 6,678 (n=21)
rpm_halved	stand-in	20	296	4.1	1,536 / 6,914 / 6,914 (n=14)	1,536 / 6,914 / 6,914 (n=14)
steady	stand-in	17	236	4.4	1,577 / 5,076 / 5,076 (n=15)	1,577 / 5,076 / 5,076 (n=15)

Table 2b: Outcomes

Scenario	Real or stand-in	Succeeded %	Rate-limited %	Timed out or given up %	Retries per request	Peak queue length	Tokens per answer	Cost per 1,000 answers
burst	stand-in	25.0%	75.0%	0.0%	0.00	1	1,892	$0.17 at list price ($0 on free tier)
daily_exhausted	stand-in	35.3%	64.7%	0.0%	0.00	1	2,102	$0.20 at list price ($0 on free tier)
overload	stand-in	30.6%	67.7%	0.0%	0.00	1	2,030	$0.20 at list price ($0 on free tier)
overload_naive_retry	stand-in	33.9%	66.1%	0.0%	3.77	0	2,111	$0.20 at list price ($0 on free tier)
rpm_halved	stand-in	70.0%	30.0%	0.0%	0.10	1	2,070	$0.19 at list price ($0 on free tier)
steady	stand-in	88.2%	11.8%	0.0%	0.00	1	2,097	$0.20 at list price ($0 on free tier)

Table 3: Retrieval settings

Setting (piece size, pieces fetched, re-rank)	Questions asked	Correct and cited %	Wrong %	Correctly refused %	Tokens per answer	First token p50 / p95 (ms)	Total time p50 / p95 (ms)
384 tokens, 4 pieces, hybrid, re-rank no	18	22.2%	50.0%	27.8%	2,124	945 / 3,518	945 / 3,518